

Filière: Transformation digitale et Intelligence artificielle

Module : DEEP LEARNING

Titre :

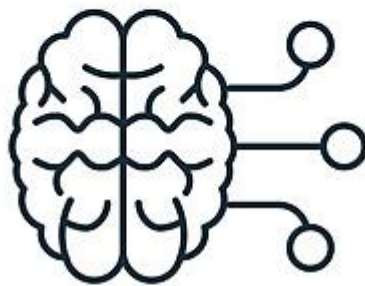
Régularisation et Hyper-paramètres

Realise par :

✓ MAZGOUT Rim

Encadré par :

✓ Pr. CHERRADI Mohamed



DEEP LEARNING

Année universitaire : 2025/2026

Table des matières

1	Introduction	2
2	Hyper-paramètres d'Optimisation	3
2.1	Pas d'apprentissage (Learning Rate — α)	3
2.2	Nombre d'itérations et d'époques	3
2.3	Méthodes d'optimisation	4
2.3.1	SGD — Stochastic Gradient Descent	4
2.3.2	Adam — Adaptive Moment Estimation	4
2.4	Batch Size	4
2.5	Régularisation — Paramètre λ	5
2.5.1	Régularisation L2 (Ridge)	5
2.5.2	Régularisation L1 (Lasso)	5
3	Hyper-paramètres d'Architecture	6
3.1	Nombre de couches (Profondeur)	6
3.2	Nombre de neurones par couche (Largeur)	6
3.3	Types de couches	6
3.3.1	FC — Fully Connected (Dense)	6
3.3.2	CNN — Convolutional Neural Network	6
3.3.3	RNN — Recurrent Neural Network	7
3.3.4	LSTM — Long Short-Term Memory	7
3.3.5	GRU — Gated Recurrent Unit	7
3.4	Fonctions d'activation	7
3.5	Dropout	7
3.6	Paramètres spécifiques aux CNNs	8
3.6.1	Taille et type de filtre	8
3.6.2	Stride (Pas de convolution)	8
3.6.3	Padding	8
3.6.4	Pooling	8
3.7	Transfer Learning et architectures spécifiques	8
3.7.1	ResNet (Residual Network)	9
3.7.2	VGG (Visual Geometry Group)	9
4	Conclusion	10

Introduction

Dans le domaine de l'apprentissage automatique, et plus particulièrement des **réseaux de neurones**, les **hyper-paramètres** jouent un rôle fondamental dans les performances et la capacité de généralisation d'un modèle.

Définition : Un **hyper-paramètre** est un paramètre dont la valeur est fixée *avant* l'entraînement du modèle, contrairement aux paramètres (poids et biais) qui sont appris automatiquement durant l'entraînement.

On distingue deux grandes catégories d'hyper-paramètres :

- **Hyper-paramètres d'optimisation** : ils contrôlent *comment* le modèle apprend.
- **Hyper-paramètres d'architecture** : ils définissent *ce qu'est* le modèle.

Hyper-paramètres d'Optimisation

Hyper-paramètres d'Optimisation

Les hyper-paramètres d'optimisation contrôlent le processus d'entraînement : la vitesse d'apprentissage, la quantité de données traitées à chaque étape, et la méthode utilisée pour minimiser la fonction de perte.

Pas d'apprentissage (Learning Rate — α)

Définition

Le **pas d'apprentissage** (ou *learning rate*) α contrôle la taille des mises à jour appliquées aux poids du réseau à chaque itération de la descente de gradient.

La règle de mise à jour des poids est :

$$w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$$

Impact du choix de α :

Valeur de α	Conséquence
Trop grand	Divergence, le modèle n'apprend pas correctement
Trop petit	Apprentissage très lent, risque de rester bloqué dans un minimum local
Optimal	Convergence rapide et stable

Valeurs typiques : $\alpha \in \{0.1, 0.01, 0.001, 0.0001\}$

Nombre d'itérations et d'époques

- **Itération** : une seule mise à jour des poids à partir d'un *batch* de données.
- **Époque** : un passage complet sur l'ensemble du dataset d'entraînement.

$$\text{Nombre d'itérations par époque} = \frac{\text{Taille du dataset}}{\text{Batch size}}$$

Attention : Un nombre d'époques trop élevé entraîne du **surapprentissage (overfitting)**, où le modèle mémorise les données au lieu de généraliser.

Méthodes d'optimisation

SGD — Stochastic Gradient Descent

Définition

La **descente de gradient stochastique** (SGD) met à jour les poids à partir d'un seul exemple (ou d'un mini-batch) à la fois, au lieu de tout le dataset.

Mise à jour :

$$w \leftarrow w - \alpha \cdot \nabla_w L(x_i, y_i)$$

Avantages : Simple, faible coût mémoire.

Inconvénients : Convergence bruyante, sensible au choix de α .

Adam — Adaptive Moment Estimation

Définition

Adam est un algorithme d'optimisation adaptatif qui combine *Momentum* et *RMSPprop*. Il adapte automatiquement le taux d'apprentissage pour chaque paramètre.

Formules de mise à jour :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1^{\text{er}} \text{ moment}) \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2^{\text{ème}} \text{ moment}) \quad (2)$$

$$w \leftarrow w - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \quad (3)$$

Valeurs par défaut : $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Avantages : Rapide, robuste, très utilisé en pratique.

Méthode	Adaptive	Momentum	Utilisation
SGD	Non	Non	Basique
SGD + Momentum	Non	Oui	Intermédiaire
RMSProp	Oui	Non	RNN
Adam	Oui	Oui	Recommandé

Batch Size

Définition

Le **batch size** est le nombre d'exemples d'entraînement utilisés pour calculer le gradient avant chaque mise à jour des poids.

Type	Batch Size	Caractéristiques
Batch GD	Tout le dataset	Stable, lent, coûteux en mémoire
Mini-batch GD	32, 64, 128, 256	Compromis idéal (le plus utilisé)
SGD	1	Rapide, bruité

Régularisation — Paramètre λ

Définition

La **régularisation** est une technique qui pénalise la complexité du modèle pour éviter le surapprentissage en ajoutant un terme à la fonction de perte.

Régularisation L2 (Ridge)

$$L_{L2} = L + \lambda \sum_i w_i^2$$

Pénalise les poids trop grands. Favorise des poids petits et répartis.

Régularisation L1 (Lasso)

$$L_{L1} = L + \lambda \sum_i |w_i|$$

Favorise la **sparsité** : pousse certains poids exactement à zéro (sélection de features).

Critère	L1 (Lasso)	L2 (Ridge)
Pénalité	$ w $	w^2
Sparsité	Oui	Non
Robustesse	Moins	Plus
Sélection de features	Oui	Non

Hyper-paramètres d'Architecture

Hyper-paramètres d'Architecture

Les hyper-paramètres d'architecture définissent la structure même du réseau de neurones : sa profondeur, sa largeur, le type de ses couches et les mécanismes internes qui le composent.

Nombre de couches (Profondeur)

Définition

Le **nombre de couches** (depth) détermine la profondeur du réseau. Plus le réseau est profond, plus il peut apprendre des représentations abstraites et complexes.

- Réseau peu profond : 1-2 couches cachées → tâches simples
- Réseau profond (Deep Learning) : 10, 50, 100+ couches → tâches complexes

Problème du vanishing gradient : Dans les réseaux très profonds, les gradients peuvent devenir extrêmement petits, rendant l'apprentissage des premières couches très difficile.

Nombre de neurones par couche (Largeur)

Le nombre de neurones par couche détermine la **capacité de représentation** de chaque couche.

- Trop peu de neurones → *underfitting* (modèle trop simple)
- Trop de neurones → *overfitting* + coût computationnel élevé

Types de couches

FC — Fully Connected (Dense)

Chaque neurone est connecté à tous les neurones de la couche précédente.

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

Usage : Classification, régression, couches de sortie.

CNN — Convolutional Neural Network

Applique des filtres de convolution pour extraire des caractéristiques locales.

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Usage : Images, données spatiales, vision par ordinateur.

RNN — Recurrent Neural Network

Possède une mémoire interne grâce à des connexions récurrentes.

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Usage : Séquences temporelles, texte, audio.

LSTM — Long Short-Term Memory

Variante des RNN qui gère les **dépendances à long terme** grâce à des portes (gates).

- **Forget gate** : décide quoi oublier
- **Input gate** : décide quoi mémoriser
- **Output gate** : décide quoi produire

GRU — Gated Recurrent Unit

Version simplifiée du LSTM avec seulement deux portes (reset et update). Plus rapide à entraîner, performances similaires au LSTM.

Fonctions d'activation

Définition

Une **fonction d'activation** introduit la **non-linéarité** dans le réseau, permettant d'apprendre des relations complexes.

Fonction	Formule	Usage typique
ReLU	$\max(0, x)$	Couches cachées (par défaut)
Sigmoid	$\frac{1}{1 + e^{-x}}$	Sortie binaire
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	RNN, LSTM
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	Sortie multi-classes
Leaky ReLU	$\max(0.01x, x)$	Alternative à ReLU

Dropout

Définition

Le **dropout** est une technique de régularisation qui désactive aléatoirement une fraction p des neurones durant l'entraînement pour éviter la co-adaptation.

- Durant l'entraînement : chaque neurone est désactivé avec probabilité p
- Durant l'inférence : tous les neurones sont actifs, poids multipliés par $(1 - p)$
- **Taux typiques** : $p \in \{0.2, 0.3, 0.5\}$

Un dropout trop élevé ($p > 0.7$) peut empêcher le réseau d'apprendre correctement.

Paramètres spécifiques aux CNNs

Taille et type de filtre

- Filtre 3×3 : capture les détails locaux fins
- Filtre 5×5 : capture des motifs plus larges
- Filtre 1×1 : réduction de dimensionnalité

Stride (Pas de convolution)

Le **stride** est le déplacement du filtre à chaque étape.

- Stride = 1 : sortie de même taille (avec padding)
- Stride = 2 : réduit la taille de moitié (sous-échantillonnage)

Padding

- **Same** : ajoute des zéros pour conserver la taille d'entrée
- **Valid** : pas de padding, la taille diminue

$$\text{Taille de sortie} = \frac{N - F + 2P}{S} + 1$$

où N = taille d'entrée, F = taille du filtre, P = padding, S = stride.

Pooling

Type	Opération	Usage
MaxPooling	Garde la valeur maximale	Réduction, robustesse aux translations
AvgPooling	Calcule la moyenne	Lissage, features globales
GlobalAvgPooling	Moyenne sur toute la carte	Avant couche FC finale

Transfer Learning et architectures spécifiques

Définition

Le **transfer learning** consiste à réutiliser un modèle pré-entraîné sur une grande base de données (ex : ImageNet) et à l'adapter à une nouvelle tâche.

ResNet (Residual Network)

- Introduit les **connexions résiduelles** (skip connections)
- Résout le problème du vanishing gradient
- $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$
- Variantes : ResNet-50, ResNet-101, ResNet-152

VGG (Visual Geometry Group)

- Architecture simple : empilement de couches Conv 3×3
- Très utilisé comme baseline
- Variantes : VGG-16, VGG-19

mainblue !20 Architecture	Profondeur	Innovation	Paramètres
VGG-16	16 couches	Filtres 3×3	~138M
ResNet-50	50 couches	Skip connections	~25M
ResNet-152	152 couches	Skip connections	~60M

Conclusion

Synthèse : Le choix des hyper-paramètres est l'une des étapes les plus critiques dans la conception d'un réseau de neurones. Il n'existe pas de valeurs universelles : chaque problème nécessite une exploration et un ajustement soigneux, souvent via des techniques comme la *grid search*, le *random search* ou l'optimisation bayésienne.

Hyper-paramètres d'optimisation	Hyper-paramètres d'architecture
Learning rate α	Nombre de couches
Nombre d'époques	Nombre de neurones par couche
Méthode (SGD, Adam...)	Type de couche (FC, CNN, RNN...)
Batch size	Fonction d'activation
Régularisation L1/L2 (λ)	Dropout, Pooling, Stride, Padding
	Transfer Learning (ResNet, VGG)

“Le bon choix des hyper-paramètres fait souvent la différence entre un modèle médiocre et un modèle performant.”